Pricing an option on many cores

SIMD, OpenMP and hybrid MPI for a Heston PDE solver

Zain Al-Saffi, 48008361

COSC3500 Milestone 2

October 2026

The project in one slide

Price one option by solving a PDE on a grid, then make each step parallel

The question

What is a call option worth today, when the share price and its volatility both move randomly, as in the Heston model?

Why it parallelises

No cell in a step depends on another in the same step, so cells go to AVX2 lanes, rows to cores and blocks of rows to processes.

The method

Solve the Heston PDE on a grid of share price by variance, stepping back from expiry, where every value is known, to today.

The result

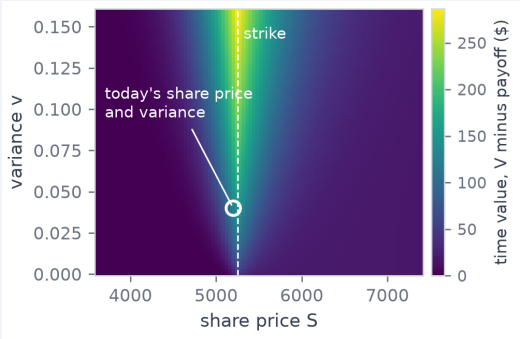
123x on one node and **147x** on two, with the same answer on every version.

The problem

The price today of the right to buy a share at a fixed price on a fixed day



At expiry the value is the payoff, max(S - K, 0).



Half way through the solve, the value above the payoff.

- Black-Scholes holds the volatility constant. Heston lets the variance move randomly too, so the value depends on S, v and t.

The equation

The value $V(S, v, t)$ solves one PDE, and each term has a plain meaning

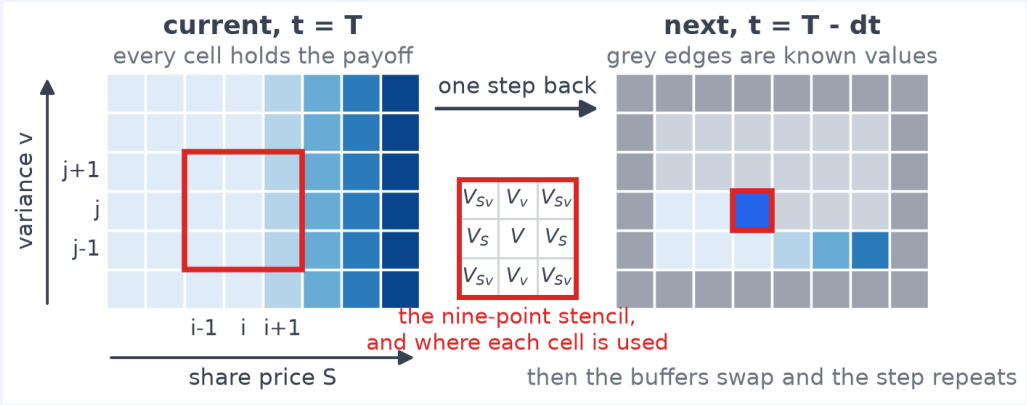
$$V_t + \tfrac{1}{2} v S^2 V_{SS} + \rho \xi v S\, V_{Sv} + \tfrac{1}{2} \xi^2 v\, V_{vv} + (r - q) S\, V_S + \kappa(\theta - v)\, V_v - rV = 0$$

$\frac{1}{2} v S^2 V_{SS}$	Diffusion in the share price, from its random movement.
$\rho \xi v S V_{Sv}$	The correlation between the share and its variance. With $\rho = -0.7$ a falling share raises volatility.
$\frac{1}{2} \xi^2 v V_{vv}$	Diffusion in the variance, scaled by the volatility of volatility ξ.
$(r - q) S V_S$	The share drifts at the risk-free rate less the dividend yield.
$\kappa(\theta - v) V_v$	Mean reversion of the variance towards its long-run level θ at rate κ.
$-rV$	Discounting, because money later is worth less than money now.

- On the grid each derivative becomes a difference of neighbouring cells, and one step adds Δt times these six terms to the old value.

One step on the grid

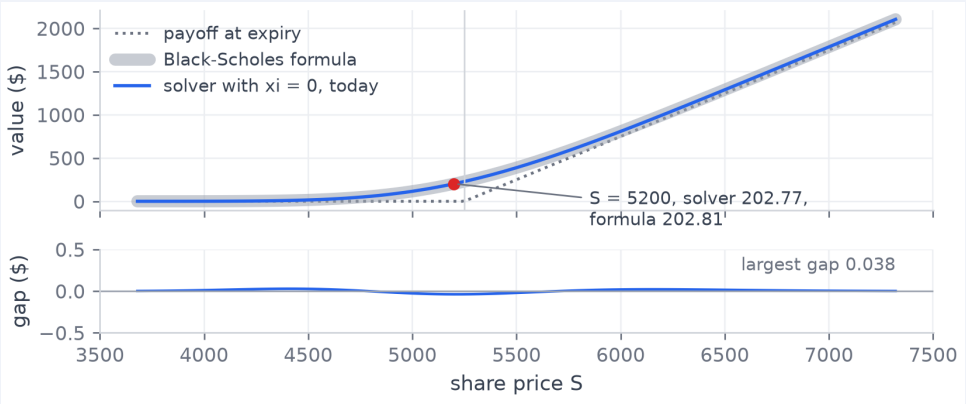
From expiry T to $T - \Delta t$, every new cell from nine old ones



- A step reads only the old buffer and writes only the new one, so the new cells can be computed in any order.

Checking the answer

With $\xi = 0$ and $\rho = 0$ the variance never moves and Heston becomes Black-Scholes



- With its full stencil unchanged, the solver is within 4 cents of the closed form at today's price. Every benchmark job runs this check first.

The data structure

Two flat arrays of doubles, read one, write the other, then swap

the grid include/grid.h:36-38, 60-62

```
// Position of node (stock_i, var_j) in the flat buffer.
// Rows are stored one after another in memory.
int index(int stock_i, int var_j) const { return var_j *
    num_stock_nodes_ + stock_i; }
// ...
// The two buffers.
std::vector<double> current_;
std::vector<double> next_;
```

16 KiB

one row of 2048 doubles

8 MiB

one buffer at 2048 × 512

16 MiB

both buffers

- The share price axis is contiguous, so the inner loop reads memory in order, and a step in variance is a stride of one row.

How every number was measured

One exclusive rangpur r730 compute node, through sbatch

- Dual Xeon E5-2670 v3, 2 x 12 cores, 30 MiB L3 per socket, g++ 8.5, Open MPI 4.1.
- `--exclusive` on a named node, so no other job shares it.
- Median of 5 repetitions, with the threads pinned to cores.
- The tests run first in the same job, and the price is printed so the compiler cannot delete the loop.

A first round was discarded

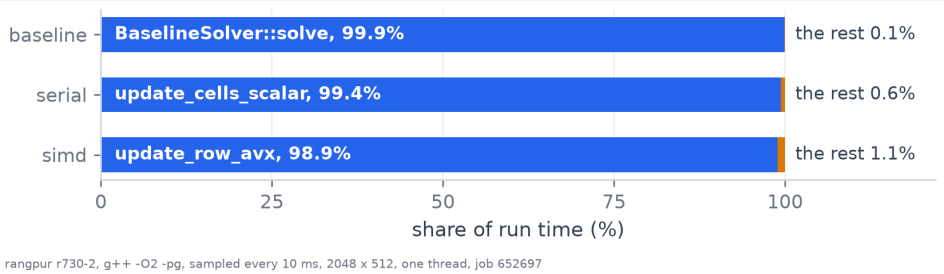
Short runs leave cells below 2.2×10^{-308}, which this CPU handles slowly. How many there were changed with the grid, so the rungs ran in different regimes. Every sweep was rerun with those values flushed to zero.

- Rates are million cell updates per second on the 2048 by 512 grid unless a slide says otherwise.

Denormal numbers, D. Goldberg, What every computer scientist should know about floating-point arithmetic, *ACM Computing Surveys* 23(1), 5 to 48, 1991.

Where the time goes

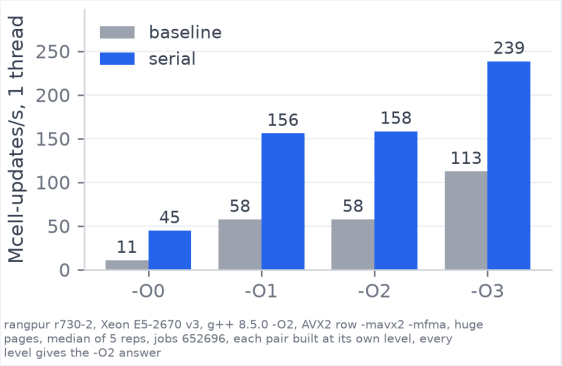
gprof flat profile, one thread, 2048 x 512



- One function takes 99 percent of the time in every single-core rung, the update of the interior rows, so every rung changes that function.

What the compiler does alone

The baseline and serial rungs built at -O0 to -O3, the same answer at every level



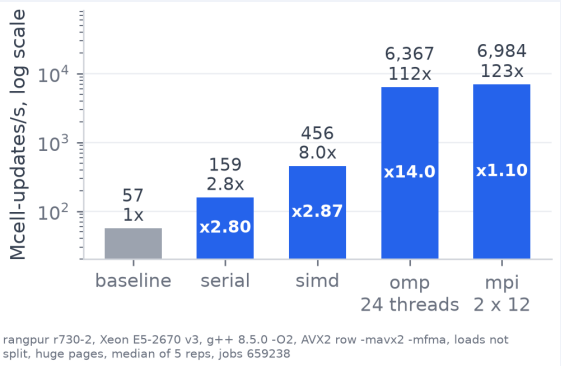
rangpur r730-2, Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, huge pages, median of 5 reps, jobs 652696, each pair built at its own level, every level gives the -O2 answer

- The baseline goes from 11 at -O0 to 58 at -O2.
- -O3 vectorises with two-lane SSE, which doubles the baseline and gives serial 1.5x.
- Every benchmark builds at -O2, so each rung measures the change written by hand.

- At -O3 the compiler takes the baseline to 113, still below the serial rung's 159 at -O2.

The ladder

Each rung adds one technique and must give the baseline's answer before it is timed



baseline	the original loop, with divisions inside it
serial	divisions turned into multiplications
simd	four cells per AVX2 instruction
omp	the rows shared between 24 cores
mpi	one process per socket, edge rows exchanged

- The number inside each bar is the gain over the rung below. The total on one node is **123x** the baseline.

The serial rung

Divisions in the loop replaced by multiplications, worked out once per solve

before, baseline src/solver_baseline.cpp:68-69

```
double dV_dS = (east - west) / (2.0 * grid.stock_spacing());
double d2V_dS2 = (east - 2.0 * centre + west) / (grid.stock_spacing() *
    grid.stock_spacing());
```

once per solve src/stencil.cpp:28-29

```
inv_two_ds = 1.0 / (2.0 * stock_spacing);
inv_ds_sq = 1.0 / (stock_spacing * stock_spacing);
```

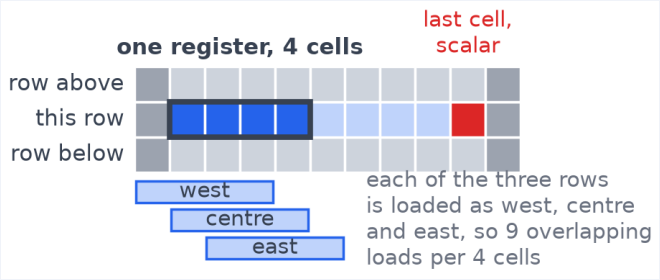
after, serial src/stencil.cpp:107-108

```
double dV_dS = (east - west) * inv_two_ds;
double d2V_dS2 = (east - 2.0 * centre + west) * inv_ds_sq;
```

- The compiler may not make this change, since the rounding can differ in the last bit. By hand it gives 159 million updates a second, **2.8x**.

SIMD, four cells per instruction

Every cell in a row runs the same formula on neighbours that sit next to each other in memory



- A 256 bit AVX2 register holds four doubles.
- Four neighbouring cells run the same formula, so one instruction updates all four.
- Each group of four cells needs nine overlapping 256 bit loads.
- The last zero to three cells of a row are computed one at a time.

- The serial rung already ran at 96 percent of the scalar peak, so the next gain came from vector instructions.

SIMD code, before and after

The serial loop and the AVX2 loop, line for line

before, serial, one cell per iteration, comments left out src/stencil.cpp:93-98, 122-123

```
for (int stock_i = first_stock; stock_i < end_stock; ++stock_i) {
    double centre = row_centre[stock_i];
    double east = row_centre[stock_i + 1];
    double west = row_centre[stock_i - 1];
// ...
    out_row[stock_i] = centre + dt * change;
}
```

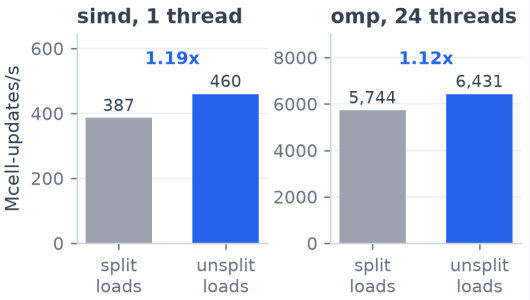
after, simd, four cells per iteration, comments left out src/stencil_avx.cpp:55-60, 108-112

```
while (stock_i + 3 <= last_interior) {
    __m256d centre = _mm256_loadu_pd(row_centre + stock_i);
    __m256d east = _mm256_loadu_pd(row_centre + stock_i + 1);
    __m256d west = _mm256_loadu_pd(row_centre + stock_i - 1);
// ...
    __m256d new_value = _mm256_fmadd_pd(dt, change, centre);
    _mm256_storeu_pd(out_row + stock_i, new_value);
    stock_i = stock_i + 4;
}
```

- Each load now reads four doubles and the update is one fused multiply-add, for 456 million updates a second, **2.9x** serial.

The compiler was splitting the loads

Experiments E2 and E2b, the same source built two ways



rangpur r730-2, Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, huge pages, median of 5 reps, jobs 657421, 657422, 657498, 657499, 2048 x 512

g++ 8.5, -mavx2 -mfma, each load split into two 128 bit loads

```
vmovupd (%r10,%rax,1),%xmm3
vmovupd -0x8(%r10,%rax,1),%xmm0
vinsertf128 $0x1,0x10(%r10,%rax,1),%ymm3,%ymm6
vinsertf128 $0x1,0x8(%r10,%rax,1),%ymm0,%ymm9
```

with -mno-avx256-split-unaligned-load, one load

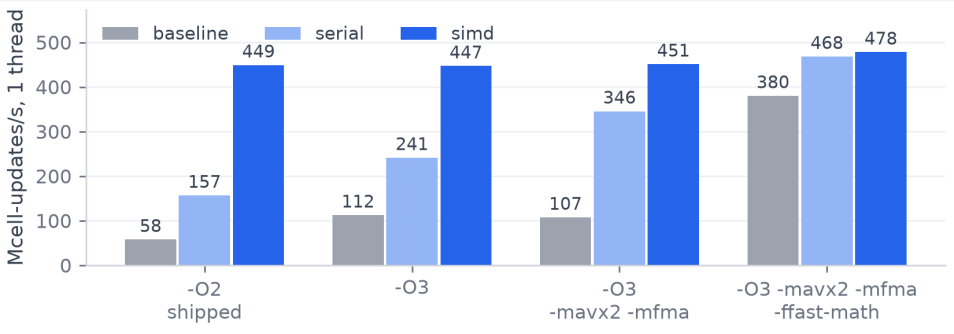
```
vmovupd 0x8(%r10,%rax,1),%ymm6
vmovupd (%r10,%rax,1),%ymm4
```

With only `-mavx2`, g++ 8.5 tunes for a generic x86 and compiles every unaligned 256 bit load as two 128 bit loads.

- Two flags make every unaligned load one 256 bit load, with the same answer and **1.19x** the speed on one core. Every number in this video uses them.

Could the compiler have written it?

Experiment E1, the whole program rebuilt under stronger flags, one thread

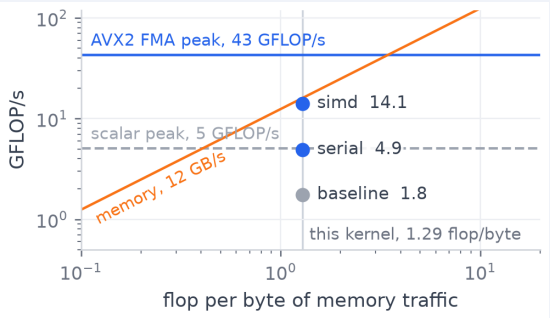


rangpur r730-2, Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, loads not split, huge pages, median of 5 reps, jobs 659248, 659249, whole program rebuilt with each set of flags

- Given AVX2, g++ reaches 0.77x of the hand-written row. -ffast-math rewrites the divisions too, but drops the IEEE rules for the whole program.

SIMD on the roofline

Measured ceilings of one r730 core



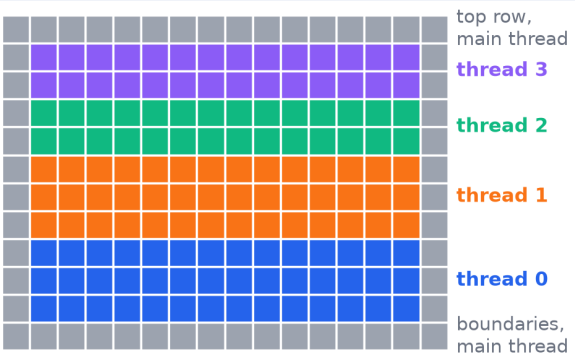
- The kernel does 31 flops for every 24 bytes it moves, so every rung sits on the grey vertical line.
- The flat lines are compute ceilings, and the sloped line is the memory bandwidth limit.
- The serial rung sits on the scalar peak.

- simd does 14 GFLOP/s, a third of the AVX2 ceiling and just under the one-core memory roof. On 4096 x 1024 memory slows it by 14 percent.

S. Williams, A. Waterman and D. Patterson, Roofline: an insightful visual performance model for multicore architectures, *Communications of the ACM* 52(4), 65 to 76, 2009.

OpenMP, one block of rows per thread

The rows of each step are shared between the cores of one node



- `schedule(static)` gives each thread one contiguous block of rows, the same block every step.
- Every thread reads the old buffer and writes only its own rows, so no locks are needed.
- The loop ends with an implicit barrier. Then the main thread updates the boundaries and swaps the buffers.

- The code change is one pragma above the row loop.

OpenMP code

The SIMD rung's row loop, and the same loop with one pragma

before, simd src/solver_simd.cpp:47-50

```
// Update every interior row, four cells at a time.
for (int var_j = 1; var_j < num_variance_nodes - 1; ++var_j) {
    update_row_avx(constants, current, next, var_j);
}
```

after, omp src/solver_omp.cpp:43-48

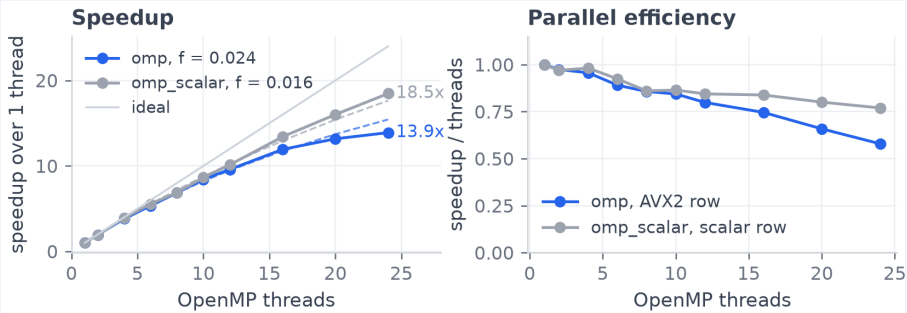
```
// Split the interior rows between the threads, one equal band each.
// Threads only read current and only write their own rows of next, so
// no locks are needed.
#pragma omp parallel for schedule(static) default(none) \
    shared(constants, current, next, num_variance_nodes)
for (int var_j = 1; var_j < num_variance_nodes - 1; ++var_j) {
    update_row_avx(constants, current, next, var_j);
}
```

- On 24 cores, 6367 million updates a second, **14.0x** the SIMD rung.

`default(none)` makes a forgotten shared variable a compile error.

Strong scaling

The same 2048 x 512 grid on 1 to 24 threads, with Amdahl's law $1/S = f + (1-f)/p$ fitted



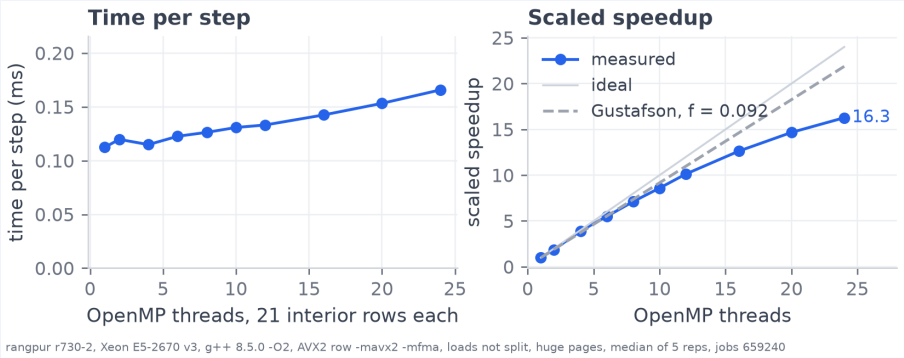
rangpur r730-2, Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, loads not split, huge pages, median of 5 reps, jobs 651297, 659239

- 13.9x on 24 threads, a serial fraction $f = 2.4$ percent. The scalar row reaches 18.5x, because each of its threads needs data at a third of the rate.

Amdahl's law, COSC3500 lecture 3. Fitting f to measured speedups, A. H. Karp and H. P. Flatt, Measuring parallel processor performance, *Communications of the ACM* 33(5), 539 to 543, 1990.

Weak scaling

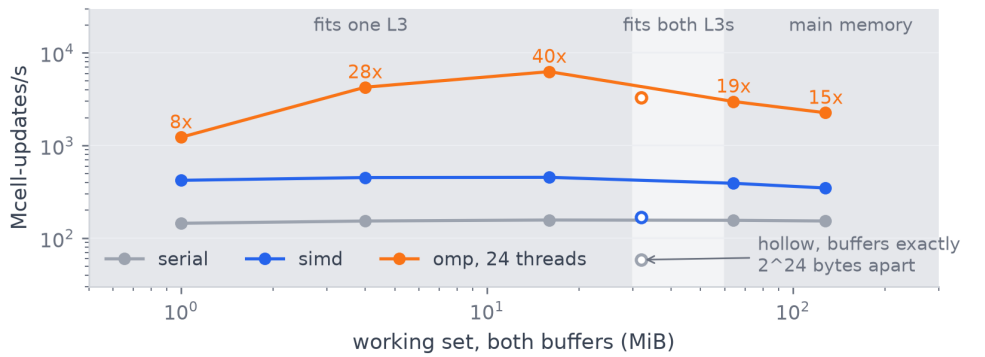
21 rows per thread, the grid grows with the threads, Gustafson's law $S = f + (1 - f)N$



- The step time grows from 0.11 to 0.17 ms, a scaled speedup of 16 on 24 threads, where Gustafson with $f = 0.09$ predicts 22.

J. L. Gustafson, Reevaluating Amdahl's law, *Communications of the ACM* 31(5), 532 to 533, 1988.

Grid size
1 MiB to 128 MiB of working set

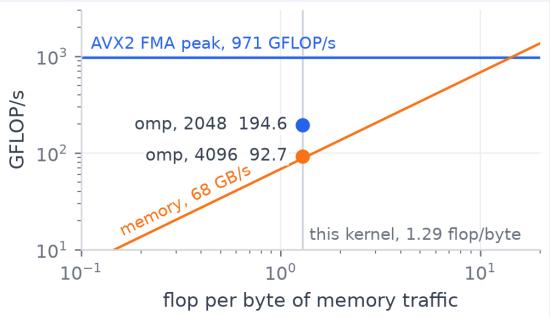


rangpur r730-2, Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, loads not split, huge pages, median of 5 reps, jobs 659241

- While both buffers fit in the two L3 caches, 24 threads run 40x serial. At 4096 by 1024 they come from main memory and the rate halves.

After threads, memory bandwidth

Measured ceilings of the whole node



rangpur r730-2, Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, loads not split, huge pages, median of 5 reps, jobs 659241

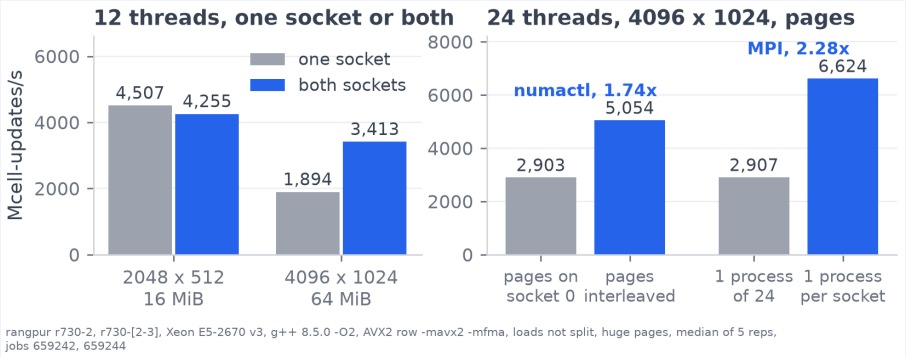
- On 2048 x 512 the grid fits in the L3 caches, so the rung runs above the memory roof.
- On 4096 x 1024 it sits on the memory roof.
- The node's AVX2 ceiling of 971 GFLOP/s is five times the small grid's rate.

- After threads the limit on the big grid is memory bandwidth, 93 GFLOP/s at a memory roof of about 88.

S. Williams, A. Waterman and D. Patterson, Roofline: an insightful visual performance model for multicore architectures, *Communications of the ACM* 52(4), 65 to 76, 2009.

Placement

Which socket the threads run on, and which socket's memory holds the pages



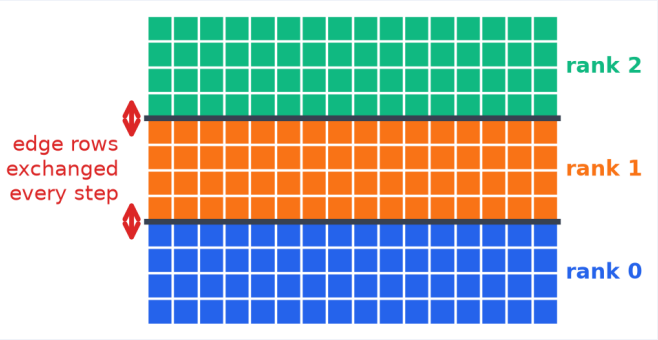
rangpur r730-2, r730-[2-3], Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, loads not split, huge pages, median of 5 reps, jobs 659242, 659244

- Two sockets give two L3 caches and two memory controllers. Each page is placed on the socket whose thread first writes it.

Pages and sockets, U. Drepper, What every programmer should know about memory, Red Hat, 2007, section 5 on NUMA.

MPI, one block of rows per process

Each process updates its own rows and receives one row from each neighbour



- Each process owns a block of rows and keeps its own copy of the grid.
- To update its first and last rows it needs one row from each neighbour, held in its copied rows.
- So every step each process sends its edge rows and receives its neighbours'.

- On one node, one process of 12 threads per socket fills its own copy of the grid, so its pages are in that socket's memory.

MPI code, the time loop

The OpenMP rung's loop, with the main changes for MPI marked

mpi, main changes from omp marked, no timers or comments src/solver_mpi.cpp:145, 156-190

```
RowRange owned_rows = rows_for_rank(num_variance_nodes, rank,
    num_ranks);
// ...
for (int step = 1; step <= num_timesteps; ++step) {
    #pragma omp parallel for schedule(static) default(none) \
        shared(constants, current, next, first_interior, end_interior)
    for (int var_j = first_interior; var_j < end_interior; ++var_j) {
        update_row_avx(constants, current, next, var_j);
    }
    if (owns_zero_row) {
        update_zero_row(constants, current, next);
    }
    apply_stock_edges(constants, next, owned_rows.begin, owned_rows.end,
        step * dt);
    if (owns_top_row) {
        copy_top_row(constants, next);
    }
    exchange_edge_rows(next, num_stock_nodes, owned_rows, rank,
        num_ranks);
    grid.swap_buffers();
}
gather_rows(grid, num_stock_nodes, num_variance_nodes, rank, num_ranks);
```

- Each process updates only its own rows and the boundaries it owns, then exchanges its edge rows before the buffers swap.

MPI code, the edge row exchange

The receive and the send with the neighbour below, then the wait

mpi, one neighbour of the edge row exchange src/solver_mpi.cpp:55-64, 86-87

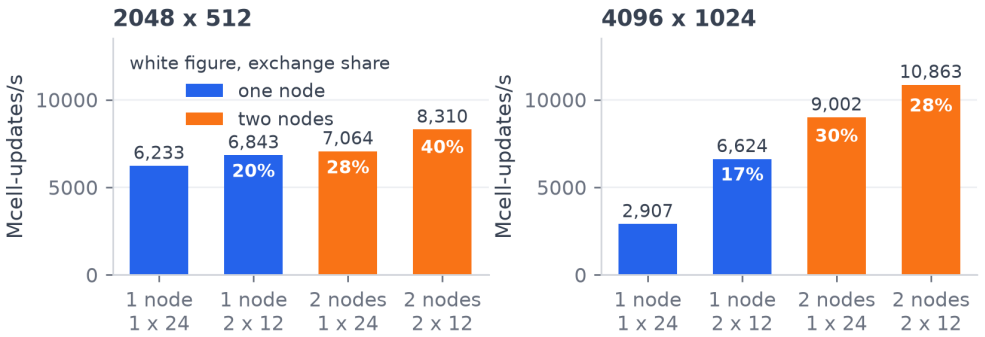
```
// The bottom process has nobody below it.
if (rank > 0) {
    // Post the receive first.
    MPI_Request receive_below;
    MPI_Irecv(row_below, num_stock_nodes, MPI_DOUBLE, rank - 1,
        message_tag, MPI_COMM_WORLD, &receive_below);
    requests.push_back(receive_below);

    MPI_Request send_below;
    MPI_Isend(first_row, num_stock_nodes, MPI_DOUBLE, rank - 1,
        message_tag, MPI_COMM_WORLD, &send_below);
    requests.push_back(send_below);
// ...
// Wait for every send and receive to finish.
MPI_Waitall((int)requests.size(), requests.data(), MPI_STATUSES_IGNORE);
```

- The receive is posted before the send, and both return immediately. `MPI_Waitall` blocks until every send and receive has completed.

Hybrid layouts

One and two nodes, two grid sizes

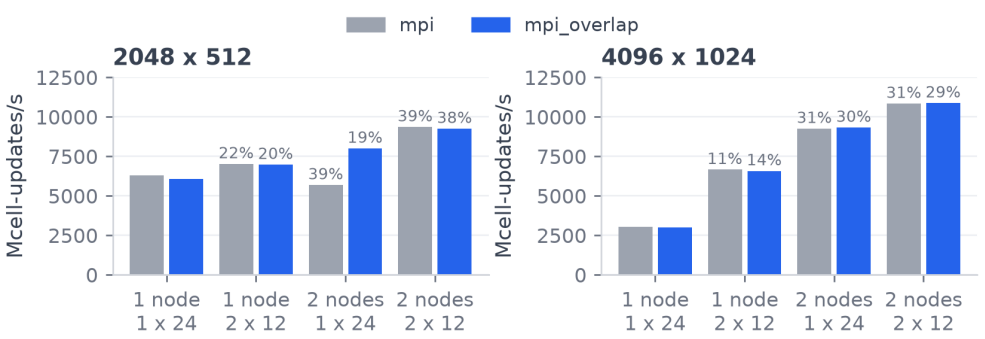


rangpur r730-[2-3], Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, loads not split, huge pages, median of 5 reps, jobs 659242

- Two nodes give **1.21x** one node on the small grid and **1.64x** on the big one. The white figure is the exchange share of a step.

Overlapping the exchange with the update

Experiment E9, the edge rows sent first while the other rows are updated

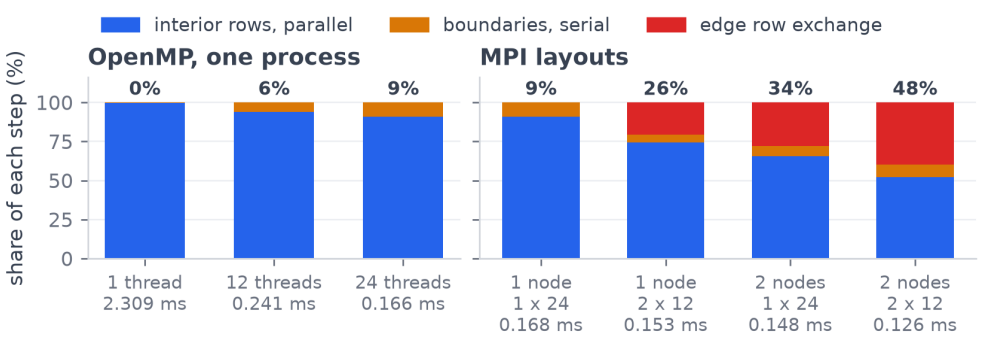


rangpur r730-[2-3], Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, loads not split, huge pages, median of 5 reps, jobs 659243, the number above each bar is the exchange share

- It helps only with one process per node, **1.41x**. Within a node Open MPI most likely transfers a 16 KiB row only during `MPI_Waitall`.

Where each step goes

Parallel rows, serial boundaries and the edge row exchange, 2048 x 512

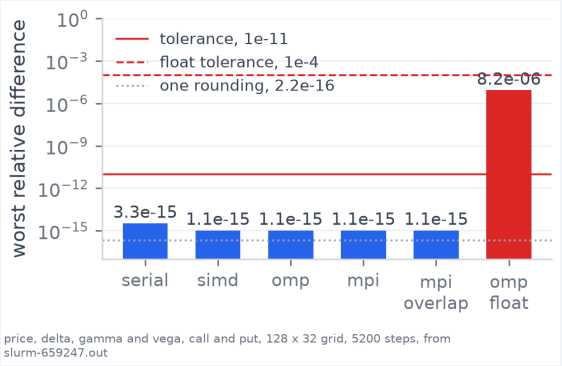


rangpur r730-2, r730-[2-3], Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, loads not split, huge pages, median of 5 reps, jobs 651297, 659239, 659242, 2048 x 512, the number above each bar is the share that is not parallel

- At 24 threads the serial boundaries take 9 percent of a step, as the main thread reads rows other cores just wrote.

Same answer on every rung

Price, delta, gamma and vega against the baseline on the test grid

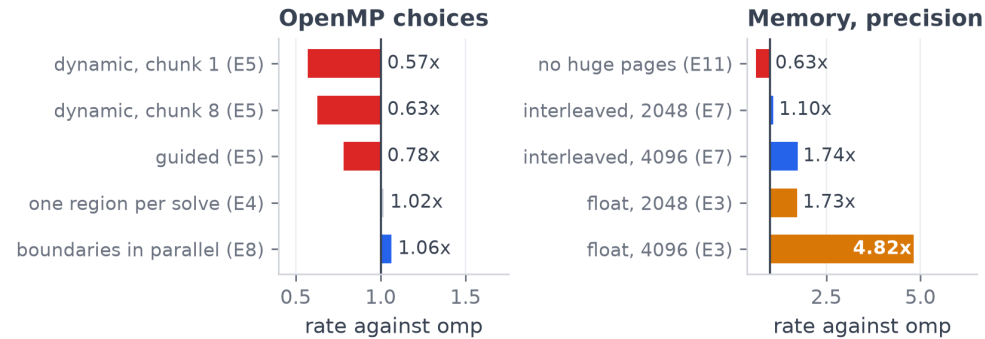


- Every rung is within 3.3×10^{-15} of the baseline, a few roundings of a double.
- OpenMP and MPI equal the SIMD rung bit for bit, because they split the same row updates and never sum across workers.
- Float, the red bar, was held to its own limit.

- No version was timed until it matched, and every benchmark job reran these tests before timing anything.

Tried and rejected

Each bar is one variant against the OpenMP rung, 24 threads



rangpur r730-2, Xeon E5-2670 v3, g++ 8.5.0 -O2, AVX2 row -mavx2 -mfma, loads not split, huge pages, median of 5 reps, jobs 659244, 659245, 659246, 659247, 24 threads

- Dynamic scheduling lost, because every row costs the same. Float was faster but moved the price by 4 cents, so it was rejected.

Rung	Mcell/s	Total
baseline	57	1x
serial	159	2.8x
simd	456	8.0x
omp, 24 threads	6367	112x
mpi, one node	6984	123x
mpi, two nodes	8310	**147x**

- Serial reached the scalar peak, so the next gain came from SIMD, and unsplit 256 bit loads added 1.19x.
- Threads took the big grid onto the memory roof of 68 GB/s.
- Across nodes the exchange is the limit, and overlapping it only helps when the library sends the row immediately.

- Of the six hypotheses four held, two nodes were efficient only on the big grid, and the serial boundary grew with the threads.

What surprised me, and what I would do next

Looking back over the project

What surprised me

- g++ split every unaligned AVX2 load into two 128 bit loads, and the two flags that stop this made simd 1.19x faster.
- Serial and simd ran up to 2.7x slower on a grid whose two buffers sit exactly 2^{24} bytes apart.
- Huge pages, denormals and page placement moved the rates as much as some optimisations did.

What I would do next

- Keep only each process's own rows in memory, so the grid can outgrow one node.
- Raise Open MPI's shared memory eager limit, to test why E9 only helped across nodes.
- Try an implicit scheme for longer timesteps, and a GPU version of the row update.

- Settings outside the code changed the results as much as some code changes, so every result file records the node, compiler and flags.

123x on one node, 147x on two, the same answer on every rung

Every number has a job id in docs/RESULTS.md

COSC3500 Milestone 2, Zain Al-Saffi

Appendix

Backup slides for questions. They are not part of the video.

Where everything is

The files to open, in the order the video meets them

`include/grid.h`	The grid, `index()` and the two buffers.
`src/solver_baseline.cpp`	Rung 0, the original loop nest with the divisions inside it. Every rung is checked against it.
`src/stencil.cpp`	The constants worked out once, the scalar row and the boundaries.
`src/stencil_avx.cpp`	The AVX2 row, the only file built with `-mavx2 -mfma`, with a scalar fallback.
`src/solver_{serial,simd,omp,mpi}.cpp`	Rungs 1 to 4, one time loop each.
`src/solver.cpp`	`make_solver`, the stability report and reading the price off the grid.
`src/solver_experiments.cpp`	Experiments E4, E5 and E8. E2 is in `solver_align.cpp`, E3 in `solver_float.cpp`, E9 in `solver_mpi.cpp`.
`tests/`	Four test programs, run by `make test` and `make test-mpi`.
`slurm/`	One job script per sweep, all sharing `bench_common.sh`.
`scripts/`	Every figure, drawn from the CSVs the jobs wrote.
`docs/RESULTS.md`	Every number, with its job id.

Two random processes

$$dS = (r - q)S\,dt + \sqrt{v}\,S\,dW_S$$
$$dv = \kappa(\theta - v)\,dt + \xi\sqrt{v}\,dW_v$$
$$dW_S\,dW_v = \rho\,dt$$

The Feynman-Kac theorem turns the expected discounted payoff into the PDE. The random terms become the second derivatives, so the program has no random number generator.

Parameter	Value
today's share price S_0	5200
strike K, expiry T	5250, 0.25 y
rate r, dividend yield q	0.045, 0.013
$v_0 = \theta$	0.04
κ, ξ, ρ	1.5, 0.35, -0.7
grid	2048 x 512

The five differences

The nine cells a step reads, and what each difference measures

nw	north	ne
west	V	east
sw	south	se

share price across, variance up

	difference	**what it measures**
V_S	(east $-$ west) $/\,2\Delta S$	how steeply value rises with share price
V_{SS}	(east $-2V+$ west) $/\,\Delta S^2$	how curved it is along the share price
V_v	(north $-$ south) $/\,2\Delta v$	how value changes with variance
V_{vv}	(north $-2V+$ south) $/\,\Delta v^2$	how curved it is along the variance
V_{Sv}	(ne $-$ nw $-$ se $+$ sw) $/\,4\Delta S\Delta v$	how the share slope changes with variance, which is why the corners are needed

The serial rung multiplies by $1/(2\Delta S)$ and the others, worked out once per solve, where the baseline divides in every cell.

The boundaries

Each edge of the grid has its own rule, all serial in the OpenMP rung

$v = 0$, the bottom row	Every diffusion term has a factor v, so only the drift terms are left. V_v looks one row up, because there is no row below.
$S = 0$, the first column	A call is worth nothing there. A put is worth the discounted strike.
largest S, the last column	A call is worth $Se^{-q\tau} - Ke^{-r\tau}$ there, and a put nothing.
the top variance row	A copy of the row below, so the value stops changing with variance at the top. It runs last.

- On one thread the boundaries are 0.4 percent of a step. At 24 threads they are 9 percent, because the main thread has to fetch rows that other cores just wrote.

The timestep limit

An explicit scheme is only stable below a largest timestep

$$\Delta t_{\text{stable}} = \frac{1}{v_{\max} S_{\max}^2 / \Delta S^2 + \xi^2 v_{\max} / \Delta v^2 + r}$$

- The far corner of the grid is the worst cell, where the diffusion is largest.
- At 2048 x 512 the limit is 3.66×10^{-7} years, so the real three-month solve takes 980000 steps.
- Every benchmark picks its maturity so that Δt is 70 percent of the limit (`maturity_for` in `bench_common.sh`).
- Above the limit the solution grows without bound and the price comes out as `nan`.

Why an explicit scheme

Each new cell comes straight from known values

- An explicit step computes each new cell from the old buffer, $V^{\text{new}} = V + \Delta t \times$ (the six terms), with no equation to solve.
- So the work is a stencil, regular and local, which splits cleanly between lanes, threads and processes.
- An implicit scheme would solve a linear system across the whole grid every step, which couples the cells and is much harder to parallelise.
- The cost of the explicit choice is the timestep limit, so the solver takes many small steps. Each step is the same regular work, which suits SIMD, threads and processes.

Reading the price and the Greeks

`extract_result` in `src/solver.cpp`, after the last step

- Today's share price and variance fall between grid nodes, so the price is interpolated bilinearly from the four cells around that point.
- The same interpolation one cell to each side gives the values the Greeks need.
- Delta and gamma are central differences in the share price, and vega a central difference in the variance, per unit of variance.
- The solver says on stderr when the quote is off a node, so an interpolated price is never mistaken for a node value.

Flops and bytes per cell

Where the roofline's x position comes from

31 flops per cell update

15 for the five differences and 16 to add up the PDE terms and step. A fused multiply-add counts as two.

24 bytes per cell update

8 to read the old value, 8 to write the new one, and 8 more because a store first reads the cache line it writes into.

The arithmetic intensity is $31/24 = 1.29$ flops per byte. The rate in GFLOP/s is the measured cell updates per second times 31. The ceilings come from `slurm/bench_peak.sh` on the same node, a scalar and an AVX2 FMA loop for compute and a triad for memory.

The AVX2 row next to the scalar row

The same sum, one cell or four

serial, one cell src/stencil.cpp:113-122

```
// The five terms of the Heston PDE.
double stock_diffusion_term = stock_diffusion * stock_price_sq[stock_i]
    * d2V_dS2;
double cross_term = cross_diffusion * stock_price[stock_i] * d2V_dS_dv;
double variance_diffusion_term = variance_diffusion * d2V_dv2;
double stock_drift_term = stock_drift * stock_price[stock_i] * dV_dS;
double variance_drift_term = variance_drift * dV_dv;

// Step back one dt from the current value.
double change = stock_diffusion_term + cross_term +
    variance_diffusion_term + stock_drift_term + variance_drift_term -
    rate * centre;
out_row[stock_i] = centre + dt * change;
```

simd, four cells src/stencil_avx.cpp:98-109

```
// Add up the five PDE terms and the discounting.
// fmadd(a, b, c) works out a times b plus c with one rounding.
__m256d change = _mm256_mul_pd(stock_diffusion_factor, d2V_dS2);
change = _mm256_fmadd_pd(cross_factor, d2V_dS_dv, change);
change = _mm256_fmadd_pd(variance_diffusion, d2V_dv2, change);
change = _mm256_fmadd_pd(stock_drift_factor, dV_dS, change);
change = _mm256_fmadd_pd(variance_drift, dV_dv, change);
change = _mm256_fnmadd_pd(rate, centre, change);

// Step back one dt and store the four new values.
__m256d new_value = _mm256_fmadd_pd(dt, change, centre);
_mm256_storeu_pd(out_row + stock_i, new_value);
```

Why SIMD is below 4x

Four lanes, 2.9x measured

- Each group of four cells needs nine overlapping 4-wide loads, plus two for the S and S^2 tables.
- The west, centre and east loads are 8 bytes apart, so at most one of them can start on a 32 byte boundary. Every unaligned load crosses a cache line on every other iteration, which moves the rate by up to 16 percent (E2b).
- At 456 million updates a second one core moves about 11 GB/s, close to the 12.4 GB/s it can stream from memory.
- The last zero to three cells of every row are computed one at a time, and each row first loads its constants into registers.

- No hardware counters were available in the jobs, so the loss cannot be split exactly between these causes.

Where the rows sit in their cache lines

Experiment E2b, one thread, 2048 x 512, with unsplit loads

Where column 1 sits	**Mcell/s**
centre load on a 32 byte boundary	477
west or east load on a 32 byte boundary	439
no load on a 32 byte boundary	412
aligned load instructions, centre aligned	477
east and west built by register shuffles	460

An aligned load instruction runs at the same speed as an unaligned one at the same address. The rate depends on whether a load crosses a cache line.

Experiment E1 in full

The whole program rebuilt under each set of flags, one thread, 2048 x 512, Mcell/s

Build	baseline	serial	simd
`-O2`, shipped	57.6	157.0	449.6
`-O3`	112.5	241.3	446.9
`-O3 -mavx2 -mfma`	107.4	345.6	451.1
`-O3 -mavx2 -mfma`, unsplit loads in the whole program	106.4	362.7	452.0
`-O3 -march=native`	105.9	266.3	455.8
`-O2 -ffast-math`	58.6	175.1	476.5
`-O3 -ffast-math`	189.5	271.5	472.7
`-O3 -mavx2 -mfma -ffast-math`	380.1	468.2	478.0
`-O3 -mavx2 -mfma -ffast-math`, unsplit loads in the whole program	376.8	399.3	478.2

The simd column always has unsplit loads, from the two flags on its own file. Every build gives the `-O2` answers to within 1.1×10^{-14}. Why `-march=native` slows serial, and why the fast-math serial rung is slower with unsplit loads, has not been found. Jobs 659248 and 659249.

Why there are no locks

`schedule(static)`, `default(none)` and the barrier

- Each thread reads only `current` and writes only its own rows of `next`. No cell is written by two threads, and no thread reads `next` during the loop.
- `schedule(static)` splits the rows into equal contiguous blocks once. Every row costs the same, so dynamic scheduling only added overhead (E5, up to 43 percent slower).
- `default(none)` makes every variable used in the loop be listed. The loop counter and the variables inside the body are private to each thread.
- The parallel loop ends with an implied barrier, so every row is done before the boundaries and the swap.

The swap

Two references bound once, still right after every swap

the swap src/grid.cpp:29-33

```cpp
// Swaps which buffer is read and which is written.
// Only the internal pointers are swapped. No numbers are copied.
void Grid::swap_buffers() {
    std::swap(current_, next_);
}
```

bound once, before the time loop src/solver_omp.cpp:36-37

```cpp
const std::vector<double>& current = grid.current();
std::vector<double>& next = grid.next();
```

`std::swap` on two vectors swaps their internal pointers and copies no numbers. The references name the two member vectors, so after the swap `current` holds what was just written.

Constants copied into local variables

Why the row starts with a block of copies

copied into locals before the loop src/stencil.cpp:66-76

```
// Copy the constants into local variables first.
// Otherwise the compiler may reload them from memory after every write.
double dt = constants.dt;
double rate = constants.rate;
double stock_drift = constants.stock_drift;
double inv_two_ds = constants.inv_two_ds;
double inv_ds_sq = constants.inv_ds_sq;
double inv_two_dv = constants.inv_two_dv;
double inv_dv_sq = constants.inv_dv_sq;
double inv_four_ds_dv = constants.inv_four_ds_dv;
int num_stock_nodes = constants.num_stock_nodes;
```

The row writes through a `double*`, and the compiler cannot prove that pointer never points into the constants. Without the copies it would reload every constant after every store. As locals they stay in registers for the whole row.

Amdahl, a fixed problem

$S(p) = 1/\big(f + (1-f)/p\big)$. Fitted by least squares to the strong scaling speedups, $f = 2.4$ percent, more than five times the 0.44 percent of boundary work on one thread. The rest is the fork and barrier every step and the shared cache bandwidth. The scalar row fits 1.6 percent, because a faster row makes those fixed costs a larger share of each step.

Karp and Flatt 1990, Gustafson 1988, as on the scaling slides.

Gustafson, a growing problem

$S(N) = f + (1-f)N$, with f the serial share of the parallel run. Measured on 24 threads $f = 0.09$, which predicts 22. The measured scaled speedup is 16.

Thread binding on the r730 nodes

Why `close` alternates sockets

- The CPUs are numbered even on socket 0 and odd on socket 1 (job 651500 printed each thread's CPU).
- `OMP_PLACES=cores OMP_PROC_BIND=close` fills CPUs 0, 1, 2, so every even thread count is split evenly over the two sockets.
- `spread` with fewer than 24 threads stays on socket 0, which is how E6 measured one socket against two.
- Unbound threads were 1.11x faster on the small grid at 24 threads, which is not explained. On the big grid the reps range from 2699 to 4009 within one job, so the comparison there is noise.

Page placement, numactl and huge pages

Where the memory of the grid comes from

- Linux places a page in the memory of the socket whose thread first writes it, which is called the first-touch policy. The grid is filled by the main thread, so every page is on socket 0.
- `numactl --interleave=all` spreads the pages over both sockets, 1.74x on the big grid (E7). Without it the runs fall into two groups, near 2800 and near 4000, so the size of the gain depends on which group they fall in.
- One MPI process per socket allocates and fills its own copy, so its pages are local, 2.28x on the big grid.
- `MALLOC_MMAP_THRESHOLD_=1048576` makes every buffer over 1 MiB come from `mmap`, so each repetition gets fresh 2 MiB huge pages. Without it only the first repetition gets them (E11, 0.63x on 24 threads).

U. Drepper, What every programmer should know about memory, Red Hat, 2007, section 5.

Which rows a process owns

`rows_for_rank`, and `MPI_Gatherv` at the end

which rows a process owns src/solver_mpi.cpp:26-40

```
static RowRange rows_for_rank(int num_rows, int rank, int num_ranks) {
    int rows_per_rank = num_rows / num_ranks;
    int extra_rows = num_rows % num_ranks;

    RowRange rows;
    if (rank < extra_rows) {
        // One of the first few processes, with one extra row.
        rows.begin = rank * (rows_per_rank + 1);
        rows.end = rows.begin + rows_per_rank + 1;
    } else {
        // Skip past the processes that took an extra row.
        rows.begin = extra_rows * (rows_per_rank + 1) + (rank -
            extra_rows) * rows_per_rank;
        rows.end = rows.begin + rows_per_rank;
    }
    return rows;
```

When the rows do not divide evenly the first processes take one more row each. At the end `MPI_Gatherv` collects the blocks, with one count and offset per process, outside the timed steps.

E9 in code

The edge rows first, then the messages, then the other rows

E9, inside the parallel region src/solver_mpi.cpp:312-327

```
// Both rows to send are finished once every thread reaches this line.
#pragma omp barrier

// The master thread starts the messages. The other threads do not wait
// for it,
// because master, unlike single, has no barrier at its end.
#pragma omp master
{
    double post_start = omp_get_wtime();
    // A sent row needs its stock edges before it leaves.
    if (send_below_row >= 0) {
        apply_stock_edges(constants, next, send_below_row,
            send_below_row + 1, time_to_expiry);
    }
    if (send_above_row >= 0) {
        apply_stock_edges(constants, next, send_above_row,
            send_above_row + 1, time_to_expiry);
    }
    post_edge_rows(next, num_stock_nodes, owned_rows, rank, num_ranks,
        requests);
```

`barrier` makes every thread finish the rows the neighbours need. `master` lets one thread post the messages, and unlike `single` it has no barrier at its end.

Why E9 only helped across nodes

How Open MPI 4.1.1 moves a message, read from `ompi_info` on rangpur

- Between nodes it uses TCP, whose eager limit is 64 KiB, so a 16 KiB row is sent as soon as `MPI_Isend` is called.
- Between processes on one node it uses shared memory, whose eager limit is 4 KiB. A 16 KiB row uses the rendezvous protocol and is only transferred once the receiver is inside an MPI call.
- There is no progress thread, so that call is `MPI_Waitall` at the end of the step, and the transfer does not overlap the update.
- The only layout that gained, one process on each of two nodes, is the only one with no two processes on one node.

- This explanation fits every small grid layout but has not been tested.

Raising the shared memory eager limit above 16 KiB would test it.

The two node numbers between jobs

The same build and layouts, run minutes apart, Mcell/s

Layout	2048, job 659242	job 659243	4096, job 659242	job 659243
1 node, 1 x 24	6233	6288	2907	3045
1 node, 2 x 12	6843	7025	6624	6664
2 nodes, 1 x 24	7064	5693	9002	9262
2 nodes, 2 x 12	8310	9364	10863	10848

The two node layouts on the small grid differ between the jobs by more than the repetitions inside one job do. Every other layout agrees within 5 percent. So the two node gain on the small grid lies between 1.21x and 1.33x.

- The headline quotes job 659242, which the hybrid figure is drawn from. Job 659243 is the E9 job, and these are its `mpi` rows.

MPI started for threads

`MPI_Init_thread` with `MPI_THREAD_FUNNELED`

MPI started for the overlap rung src/main.cpp:113-117

```cpp
// Every solver runs inside MPI. The non MPI solvers run as one process.
// FUNNELED promises MPI that only the main thread calls it, even while
// OpenMP threads run.
// The mpi_overlap solver needs that, because its master thread posts
// messages inside a parallel region.
int thread_level = 0;
MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &thread_level);
```

FUNNELED tells MPI that only the main thread calls it while OpenMP threads run. The `mpi` rung calls MPI only outside parallel regions. The `mpi_overlap` rung calls it from the master thread inside one, so it checks the level and refuses to run below it.

The test that timed out

One 3-process solve of the test grid under three settings

Solver	close, on cores	passive wait	not bound
simd	1.05 s	1.04 s	0.62 s
omp	1.04 s	1.12 s	0.59 s
mpi	14.24 s	1.16 s	0.58 s
mpi_overlap	14.66 s	1.17 s	0.60 s

The bench scripts pinned threads to cores, and the MPI test inherited that, so every process put its two threads on cores 0 and 1. Waiting OpenMP threads busy-wait, so one process held the cores its neighbour needed to finish and send its row. The test now sets `OMP_PROC_BIND=false`, and validation fell from 14 to 2.5 minutes.

Denormal numbers

Flushed to zero by a library loaded before `main`

loaded with LD_PRELOAD by the bench scripts slurm/flush_denormals.c:10-14

```c
/* Runs before main. Bit 15 (0x8000) is FTZ, results too small to be a
    normal double become 0.
   Bit 6 (0x0040) is DAZ, denormal inputs are read as 0. Threads made
       later copy this setting */
__attribute__((constructor)) static void flush_denormals(void) {
    _mm_setcsr(_mm_getcsr() | 0x8040);
}
```

`__attribute__((constructor))` runs the function before `main`. It sets flush to zero and denormals are zero in the SSE control register. The bench scripts load it with `LD_PRELOAD`, so `heston` itself is unchanged.

Rung	as is	flushed
baseline	53	57
serial	124	159
simd	320	454
omp 24	4865	6484

Job 659436. Every price is identical to ten digits either way.

The 2^{24} byte address effect

A slow point in the grid size sweep, caused by where the two buffers sit in memory

- With split loads, simd on one thread ran at 169 to 183 on any grid whose two buffers sit exactly 2^{24} bytes apart, such as 2896 x 724, 1448 x 1448 and 2044 x 1026, and about 405 on the grids next to them. With unsplit loads it still runs at 169 at 2896 x 724.
- omp on 2 threads shows the same, 336 against 814. Serial is slow there too on the current code, 58 at 2896 x 724.
- omp on 24 threads ran at 6499 there in two earlier jobs, and at 3288 and 3267 in the latest two, one with each build, so the flags are not the cause.
- The grids that trigger it are known, and the hardware mechanism has not been found.

- That is why the 2896 x 724 points are hollow on the grid size figure, and why its 56x is left out.

Float instead of double

Experiment E3, faster and still rejected

- Float halves the bytes per cell and doubles the lanes per register. On one thread 1.92x simd, on 24 threads 1.73x on 2048 x 512 and 4.8x on 4096 x 1024.
- The big grid gains most, because the float buffers (32 MB) fit in the two L3 caches (60 MB) and the double buffers (64 MB) do not.
- On the reference solve the float price is 196.1256 against 196.1684 in double and 196.1692 from Fourier inversion. The 4 cent error is larger than the discretisation error of a much coarser double grid.
- Gamma moved by 11.9 percent, vega by 2.3 and delta by 0.46.

The six hypotheses

Written before the sweeps ran, then checked against them

#	Hypothesis	Outcome
1	SIMD gives 2 to 3.5x over serial	held, 2.9x
2	OpenMP scales close to linearly to 12 threads	mostly held, 9.6x on 12 threads, 80 percent efficient
3	the large grid becomes memory-bound	held, on the memory roof at 4096 x 1024
4	the serial boundary stays near 0.5 percent	overturned, 9 percent at 24 threads
5	one process per socket beats one of 24 threads	held, 1.10x and 2.28x
6	two nodes above 80 percent efficient	overturned at 2048 x 512, 61 percent. Held at 4096 x 1024, 82 percent

What the tests check

Run by `make test` and `make test-mpi` before every benchmark

`test_bs_collapse`	With $\xi = 0$ and $v_0 = \theta$ the variance never moves, and the solver must match Black-Scholes.
`test_parity`	Put-call parity, $C - P = Se^{-qT} - Ke^{-rT}$, on the full model. It holds in any model, so a gap means a bug.
`test_ladder_matches`	Every rung against the baseline within 10^{-11}, and the parallel rungs against simd to the bit, at 1, 2 and 7 threads and on 2 and 3 processes.
`test_avx_row`	The AVX2 row against the scalar row on random grid values.

- 7 threads and 3 processes do not divide the 32 test rows evenly, so the uneven split is tested too.

How the figures were made

Every figure is drawn by a script from files the jobs wrote

`scripts/m2_figures.py`	Every benchmark chart, from the sweep CSVs, the peak file, a test log and the gprof output. Each bar or point is the median of 5 repetitions in one exclusive job.
`scripts/decomposition_diagram.py`	The three diagrams of who owns which cells.
`scripts/method_figures.py`	The grid step, the Black-Scholes check and the value map, the last two from `heston --dump-every` snapshots.
`scripts/payoff_diagram.py`	The payoff.
`scripts/code_shots.py`	The code images, from the source files, with the file and lines on each.

- `make m2-figures` and `make code-shots` redraw everything. The script refuses any CSV whose host is not a compute node.

Tests before timing

The job stops unless every test passes

runs before every sweep slurm/bench_common.sh:62-67

```
build_and_validate() {
  make clean
  make all || { echo "FATAL: build failed, refusing to benchmark"; exit
      1; }
  make test || { echo "FATAL: make test failed, refusing to benchmark";
      exit 1; }
  make test-mpi || { echo "FATAL: make test-mpi failed, refusing to
      benchmark"; exit 1; }
  make avx-check || { echo "FATAL: make avx-check failed, refusing to
      benchmark"; exit 1; }
```

Every benchmark job builds from clean, runs `make test`, `make test-mpi` and `make avx-check`, and only then times anything.

Where a number came from

The header every benchmark CSV carries, written by the job

results/bench_m2_ladder_659238.csv, header written by the job

```
# sweep_name=m2_ladder
# host=r730-2.compute.eait.uq.edu.au
# nodes=r730-2
# date=2026-10-14T11:03:04+10:00
# jobid=659238
# compiler=g++ (GCC) 8.5.0 20210514 (Red Hat 8.5.0-28)
# mpi=mpiexec (OpenRTE) 4.1.1
# flags=-std=c++17 -O2 -Wall -Wextra -Iinclude
# avx_flags=-mavx2 -mfma -mno-avx256-split-unaligned-load -mno-avx256-split-unaligned-store
# reps=5
# page_regime=MALLOC_MMAP_THRESHOLD_=1048576, THP [always] madvise never
# omp_places=cores omp_proc_bind=close
# flush_denormals=1 (FTZ and DAZ set by slurm/flush_denormals.c through LD_PRELOAD, unless a row label says otherwise)
# cpu=Intel(R) Xeon(R) CPU E5-2670 v3 @ 2.30GHz
# Thread(s) per core:  1
# Core(s) per socket:  12
# Socket(s):           2
# NUMA node(s):        2
# NUMA node0 CPU(s):   0,2,4,6,8,10,12,14,16,18,20,22
# NUMA node1 CPU(s):   1,3,5,7,9,11,13,15,17,19,21,23
# cache_L1d cache:           32K
# cache_L1i cache:           32K
# cache_L2 cache:            256K
# cache_L3 cache:            30720K
```